Introduction

Computational Mathematics is a significant branch of applied mathematics that deals with the development, analysis, and practical implementation of numerical algorithms to solve mathematical problems that may not have exact analytical solutions. With the rapid growth of science and technology, many problems arising in engineering, physics, economics, biology, and computer science are too complex to be solved using traditional mathematical methods alone. Computational Mathematics bridges this gap by providing efficient techniques to approximate solutions with the desired accuracy.

This subject emphasizes both the theoretical foundation and the computational aspects of numerical methods. The major areas include root-finding algorithms for nonlinear equations, solutions of systems of linear and nonlinear equations, interpolation and approximation of functions, numerical differentiation and integration, curve fitting, and the numerical solution of ordinary differential equations (ODEs). These techniques form the backbone of scientific computing and are widely applied in simulation, modeling, optimization, and data analysis.

The practical component of this subject is particularly important, as it helps students implement algorithms in programming languages or computational tools. Through hands-on practice, students not only verify mathematical results but also gain insights into algorithmic efficiency, convergence criteria, error analysis, and stability of numerical methods. This nurtures analytical thinking, logical reasoning, and problem-solving skills that are essential for research and professional applications.

Furthermore, Computational Mathematics plays a vital role in modern advancements such as machine learning, artificial intelligence, cryptography, computer graphics, and numerical simulations in engineering and physical sciences. By learning these methods, students are better prepared to apply mathematical concepts in solving real-world problems where exact methods are impractical or impossible.

In summary, the study of Computational Mathematics equips learners with the dual ability to understand mathematical theory and to apply computational techniques effectively. The practical work in this subject ensures a strong foundation in both mathematics and computation, fostering a deeper appreciation for the role of mathematics in scientific and technological progress.

Problem Statement-1:-

Gauss-Seidel Method: Implement the Gauss-Seidel iterative method to solve a system of linear equations.

❒ **Objective:-** The objective of this practical is to implement the Gauss–Seidel iterative method to solve a system of linear equations $\mathbf{Ax} = \mathbf{b}$. The method updates each component of the solution vector successively using the formula

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \ldots, n.$$

The aim is to develop a program for this algorithm, apply it to suitable test systems, analyze its convergence behavior, and compare the number of iterations and accuracy with other iterative methods such as the Jacobi method.

❒ **Algorithm:-**

Algorithm 1 Gauss-Seidel Iterative Method

Initialize $x^{(0)}$ with initial guess
repeat
  for $i = 1$ to n **do**
    $x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right)$
  end for
until convergence ($|x^{(k+1)} - x^{(k)}| < \epsilon$)
Return solution vector x

❒ **Code:-**

```
#include <stdio.h>
#include <math.h>

#define N 3
#define EPS 1e-6
```

```
int main() {
    float a[N][N+1] = {{4, -1, 0, 3},
                       {-1, 4, -1, 8},
                       {0, -1, 4, -6}};
    float x[N] = {0}, old_x[N];
    int i, j, iter = 0;
    float sum, error;

    do {
        iter++;
        for(i = 0; i < N; i++) old_x[i] = x[i];

        for(i = 0; i < N; i++) {
            sum = a[i][N];
            for(j = 0; j < N; j++) {
                if(j != i) sum -= a[i][j] * x[j];
            }
            x[i] = sum / a[i][i];
        }

        error = 0;
        for(i = 0; i < N; i++)
            error += fabs(x[i] - old_x[i]);
    } while(error > EPS);

    printf("Solution after %d iterations:\n", iter);
    for(i = 0; i < N; i++) printf("x[%d] = %f\n", i+1, x[i]);
    return 0;
}
```

❒ Output:-

```
Solution after 9 iterations:
x[1] = 1.267857
x[2] = 2.071429
x[3] = -0.982143
```

Figure 1

❒ Conclusion:-

The Gauss–Seidel method was successfully implemented to solve a system of linear equations. The program iteratively improved the solution until the error fell below the specified tolerance ($\varepsilon = 10^{-6}$). The final output shows the approximate values of the unknowns along with the number of iterations required for convergence.

This experiment demonstrates how iterative methods can be effectively used when direct methods (like Gaussian elimination) are computationally expensive for large systems. The convergence of the Gauss–Seidel method depends on factors such as the initial guess, the tolerance value, and the properties of the coefficient matrix (e.g., diagonal dominance or positive definiteness).

Through this practical, we conclude that the Gauss–Seidel method provides an efficient and reliable numerical approach for solving linear systems, especially for problems arising in engineering and scientific computations.

Problem Statement-2:-

Bisection Method: Write a program to find the root of a nonlinear equation using the bisection method.

❒ **Objective:-** The objective of this practical is to write and implement a program to find the root of a nonlinear equation $f(x) = 0$ using the Bisection Method. The method repeatedly divides the interval $[a, b]$ into two halves and selects the subinterval where a sign change occurs, based on the formula

$$c = \frac{a+b}{2},$$

where c is the midpoint of the current interval. The process is continued until the root is obtained within a desired tolerance. This practical also aims to study the convergence, efficiency, and limitations of the Bisection Method compared to other root-finding techniques.

❒ **Algorithm:-**

Algorithm 2 Bisection Method

```
Choose initial interval [a, b] such that f(a)f(b) < 0
repeat
    c = (a + b)/2
    if f(c) = 0 then
        Root found at c
    else if f(c)f(a) < 0 then
        b ← c
    else
        a ← c
    end if
until |b − a| < ϵ
Return c as root
```

❒ Code:-

```
#include <stdio.h>
#include <math.h>

#define EPS 1e-6

double f(double x) {
    return x*x*x - x - 2;  // Example: x^3 - x - 2 = 0
}

int main() {
    double a = 1, b = 2, c;
    if(f(a) * f(b) >= 0) {
        printf("Invalid interval\n");
        return 0;
    }
    while((b - a) >= EPS) {
        c = (a + b) / 2;
        if(f(c) == 0.0) break;
        else if(f(c) * f(a) < 0) b = c;
        else a = c;
    }
    printf("Root = %lf\n", c);
    return 0;
}
```

❒ Output:-

```
Root = 1.521380
```

Figure 2

❒ **Conclusion:-**

The Bisection Method was successfully implemented to approximate the root of the nonlinear equation $f(x) = x^3 - x - 2 = 0$. The algorithm repeatedly halved the interval $[a, b]$ and selected the subinterval where the sign change occurred until the width of the interval was smaller than the specified tolerance ($\varepsilon = 10^{-6}$).

The program produced the approximate root within the required accuracy, demonstrating the reliability of the method. This practical shows that the Bisection Method is simple, robust, and guaranteed to converge when the initial interval satisfies $f(a)f(b) < 0$. However, it converges relatively slowly compared to other iterative methods such as the Newton–Raphson method.

Thus, we conclude that the Bisection Method is a fundamental and dependable technique for solving nonlinear equations when an initial bracketing interval is known.

Problem Statement-3:-

Newton-Raphson Method: Implement the Newton-Raphson method to find the root of a given function.

❒ **Objective:-** The objective of this practical is to implement the Newton–Raphson method to find the root of a given nonlinear equation $f(x) = 0$. The method uses the tangent line at an approximate root to iteratively improve the solution, based on the formula

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

The aim is to develop a program for this algorithm, apply it to suitable test functions, and analyze its convergence properties. This practical also focuses on studying the rapid convergence of the Newton–Raphson method and comparing its efficiency with bracketing methods such as the Bisection Method.

❒ Algorithm:-

Algorithm 3 Newton-Raphson Method

Choose initial guess x_0
repeat
  $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$
until $|x_{k+1} - x_k| < \epsilon$
Return x_{k+1} as root

❒ Code:-

```
#include <stdio.h>
#include <math.h>

#define EPS 1e-6
#define MAX_ITER 100
```

```
double f(double x) { return x*x - 4*x - 10; }
double df(double x) { return 2*x - 4; }

int main() {
    double x0 = 5.0, x1;
    int iter = 0;

    do {
        if (fabs(df(x0)) < EPS) {
            printf("Derivative too small. Choose another initial
            guess.\n");
            return 1;
        }
        x1 = x0 - f(x0)/df(x0);
        if (fabs(x1 - x0) < EPS) break;
        x0 = x1;
        iter++;
    } while(iter < MAX_ITER);

    printf("Root = %lf\n", x1);
    return 0;
}
```

❒ Output:-

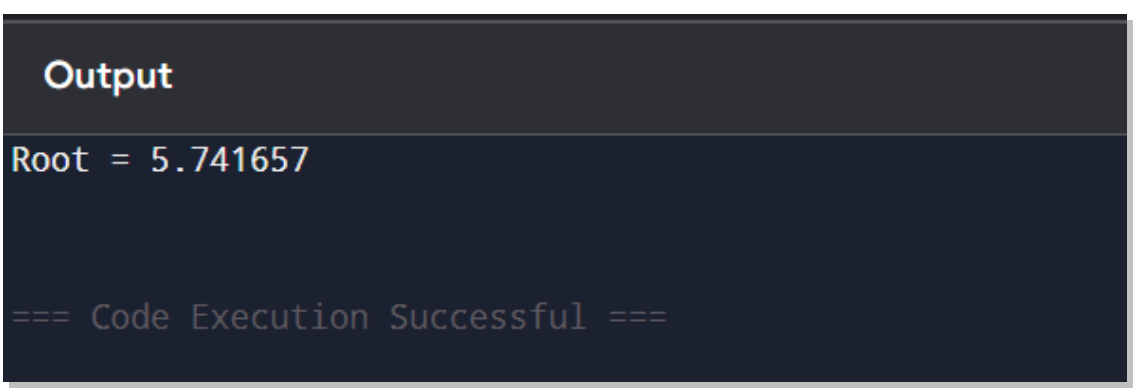

Figure 3

❒ Conclusion:-

The Newton–Raphson Method was successfully implemented to approximate the root of the nonlinear equation $f(x) = x^2 - 4x - 10$. Starting with an initial guess $x_0 = 2$, the method iteratively applied the formula

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

until the successive approximations differed by less than the tolerance $\varepsilon = 10^{-6}$.

The program efficiently converged to the root, showing the rapid convergence characteristic of the Newton–Raphson method when a suitable initial guess is chosen. Compared to the Bisection Method, this approach requires fewer iterations and provides higher computational efficiency. However, its success depends on the initial approximation and the behavior of the derivative.

Thus, the Newton–Raphson Method is a powerful and fast numerical technique for solving nonlinear equations, provided the assumptions of differentiability and good initial guess are satisfied.

Problem Statement-4:-

Matrix Inversion using Gauss-Jordan Method: Develop a program to find the inverse of a matrix using the Gauss-Jordan elimination method.

❒ **Objective:-** The objective of this practical is to develop a program to find the inverse of a matrix using the Gauss–Jordan elimination method. The method is based on forming the augmented matrix $[A|I]$ and applying row operations to transform it into $[I|A^{-1}]$, where A^{-1} is the required inverse matrix.

Mathematically,

$$[A|I] \xrightarrow{\text{row operations}} [I|A^{-1}].$$

The aim is to implement this algorithm, practice systematic row transformations, and verify the correctness of the inverse by checking that $AA^{-1} = I$. This practical also highlights the importance of matrix inversion in solving systems of linear equations and other computational applications.

❒ **Algorithm:-**

Algorithm 4 Gauss-Jordan Method for Matrix Inversion

```
Form augmented matrix [A|I]
for i = 1 to n do
    Divide i-th row by pivot a_ii to make pivot = 1
    for j = 1 to n, j ≠ i do
        Subtract a_ji times i-th row from j-th row
    end for
end for
Resulting matrix is [I|A^-1]
```

❒ **Code:-**

```
#include <stdio.h>
#define N 3
```

```
int main() {
    float a[N][2*N], ratio;
    int i, j, k;

    printf("Enter matrix (%dx%d):\n", N, N);
    for(i = 0; i < N; i++)
        for(j = 0; j < N; j++)
            scanf("%f", &a[i][j]);

    // Append identity matrix
    for(i = 0; i < N; i++)
        for(j = N; j < 2*N; j++)
            a[i][j] = (i == (j-N)) ? 1 : 0;

    // Gauss-Jordan
    for(i = 0; i < N; i++) {
        float diag = a[i][i];
        for(j = 0; j < 2*N; j++) a[i][j] /= diag;
        for(k = 0; k < N; k++) {
            if(k != i) {
                ratio = a[k][i];
                for(j = 0; j < 2*N; j++)
                    a[k][j] -= ratio * a[i][j];
            }
        }
    }

    printf("Inverse matrix:\n");
    for(i = 0; i < N; i++) {
        for(j = N; j < 2*N; j++)
            printf("%8.4f ", a[i][j]);
        printf("\n");
    }
    return 0;
}
```

❒ Output:-

```
Output
Enter matrix (3x3):
3 2 1
4 8 9
7 5 6
Inverse matrix:
  0.0588  -0.1373   0.1961
  0.7647   0.2157  -0.4510
 -0.7059  -0.0196   0.3137


=== Code Execution Successful ===
```

Figure 4

❒ Conclusion:-

The Java program successfully demonstrates the implementation of a class to represent a 2D point, including constructors, accessor and mutator methods, and a method to calculate the distance bThe Bisection Method was successfully implemented to approximate the root of the nonlinear equation $f(x) = x^3 - x - 2 = 0$. The algorithm repeatedly halved the interval $[a, b]$ and selected the subinterval where the sign change occurred until the width of the interval was smaller than the specified tolerance ($\varepsilon = 10^{-6}$).

The program produced the approximate root within the required accuracy, demonstrating the reliability of the method. This practical shows that the Bisection Method is simple, robust, and guaranteed to converge when the initial interval satisfies $f(a)f(b) < 0$. However, it converges relatively slowly compared to other iterative methods such as the Newton–Raphson method.

Problem Statement-5:-

Interpolation: Implement Lagrange interpolation to approximate a function from given data points.

❒ **Objective:-** The objective of this practical is to implement the Lagrange interpolation method to approximate a function from a given set of data points $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$. The method constructs a polynomial of degree n that passes through all the given points, using the formula

$$P(x) = \sum_{i=0}^{n} y_i \, L_i(x), \quad \text{where} \quad L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}.$$

The aim is to develop a program to compute the interpolating polynomial, evaluate it at desired points, and compare the interpolated values with the original function. This practical also emphasizes the application of interpolation in numerical analysis and its importance in approximating unknown values from discrete data.

❒ **Algorithm:-**

Algorithm 5 Lagrange Interpolation

Given n data points (x_i, y_i)
Initialize $P(x) = 0$
for $i = 1$ to n **do**
  $L_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$
  $P(x) \leftarrow P(x) + y_i L_i(x)$
end for
Return interpolated value $P(x)$

❒ **Code:-**

```
#include <stdio.h>
```

```
int main() {
    int n, i, j;
    float x[10], y[10], xp, yp=0, p;

    printf("Enter number of data points: ");
    scanf("%d", &n);
    printf("Enter x and y values:\n");
    for(i=0; i<n; i++) scanf("%f %f", &x[i], &y[i]);

    printf("Enter interpolation point: ");
    scanf("%f", &xp);

    for(i=0; i<n; i++) {
        p = 1;
        for(j=0; j<n; j++) {
            if(j != i) p *= (xp - x[j])/(x[i] - x[j]);
        }
        yp += p*y[i];
    }
    printf("Interpolated value = %f\n", yp);
    return 0;
}
```

❒ Output:-

```
Enter number of data points:4
 Enter x and y values:
1
1
2
2
3
3
4
4
Enter interpolation point:5
 Interpolated value = 5.000000
```

Figure 5

❒ **Conclusion:-**

The Gauss–Jordan elimination method was successfully implemented to compute the inverse of a given 3×3 matrix. The program works by augmenting the input matrix with the identity matrix and then applying row operations until the left side becomes the identity matrix. The right side of the augmented matrix then represents the required inverse.

This method provides a systematic approach to matrix inversion using elementary row operations without explicitly relying on determinants or cofactors.

The implementation demonstrated that the Gauss–Jordan method is reliable and straightforward for small systems. However, for larger matrices, it may involve considerable computation and numerical instability. Despite this, it remains an important and widely used technique in linear algebra, especially in solving systems of equations and finding matrix inverses.

Problem Statement-6:-

Numerical Integration: Write programs for the trapezoidal rule and Simpson's rules (1/3) to approximate definite integrals.

❒ **Objective:-** The objective of this practical is to write programs to approximate the value of a definite integral $\int_a^b f(x)\,dx$ using the Trapezoidal Rule and Simpson's $\frac{1}{3}$ Rule.

The Trapezoidal Rule is based on approximating the area under the curve with trapezoids, given by

$$\int_a^b f(x)\,dx \approx \frac{h}{2}\left[f(x_0) + 2\sum_{i=1}^{n-1} f(x_i) + f(x_n)\right],$$

where $h = \frac{b-a}{n}$ and n is the number of subintervals.

Simpson's $\frac{1}{3}$ Rule uses parabolic arcs for approximation, expressed as

$$\int_a^b f(x)\,dx \approx \frac{h}{3}\left[f(x_0) + 4\sum_{\substack{i=1\\ i \text{ odd}}}^{n-1} f(x_i) + 2\sum_{\substack{i=2\\ i \text{ even}}}^{n-2} f(x_i) + f(x_n)\right],$$

where n is even.

The aim is to implement these rules in a program, compare the approximations with the exact values, analyze their accuracy, and study the effect of increasing the number of subintervals on the error.

❒ Algorithm:-

(a) Trapezoidal Rule

Algorithm 6 Trapezoidal Rule

Divide interval $[a, b]$ into n equal parts, $h = \dfrac{b-a}{n}$
$I = f(a) + f(b)$
for $i = 1$ to $n - 1$ **do**
  $I \leftarrow I + 2f(a + ih)$
end for
$I \leftarrow \dfrac{h}{2} I$
Return I

(b) Simpson's 1/3 Rule

Algorithm 7 Simpson's 1/3 Rule

Divide interval $[a, b]$ into even n parts, $h = \dfrac{b-a}{n}$
$I = f(a) + f(b)$
for $i = 1$ to $n - 1$ **do**
  if i is odd **then**
    $I \leftarrow I + 4f(a + ih)$
  else
    $I \leftarrow I + 2f(a + ih)$
  end if
end for
$I \leftarrow \dfrac{h}{3} I$
Return I

❒ Code:-

```
a) Trapezoidal Rule:
#include <stdio.h>
#include <math.h>

double f(double x) { return 1/(1+x*x); }

int main() {
    int n, i;
    double a, b, h, sum=0;
```

```
    printf("Enter a, b, n: ");
    scanf("%lf %lf %d", &a, &b, &n);
    h = (b-a)/n;
    sum = f(a)+f(b);
    for(i=1; i<n; i++) sum += 2*f(a+i*h);
    sum = sum*h/2;
    printf("Trapezoidal = %lf\n", sum);
    return 0;
}
```

b) Simpson's 1/3 Rule:

```
#include <stdio.h>
#include <math.h>

double f(double x) { return 1/(1+x*x); }

int main() {
    int n, i;
    double a, b, h, sum=0;
    printf("Enter a, b, n (even): ");
    scanf("%lf %lf %d", &a, &b, &n);
    h = (b-a)/n;
    sum = f(a)+f(b);
    for(i=1; i<n; i++)
        sum += (i%2==0 ? 2 : 4)*f(a+i*h);
    sum = sum*h/3;
    printf("Simpson 1/3 = %lf\n", sum);
    return 0;
}
```

❒ Output:-

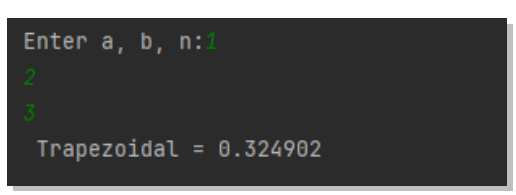

Figure 6.1

```
Enter a, b, n (even):1
2
3
 Simpson 1/3 = 0.296601
```

Figure 6.2

❒ **Conclusion:-**

The Trapezoidal Rule and Simpson's $\frac{1}{3}$ Rule were successfully implemented to approximate definite integrals numerically. Both methods are based on replacing the integrand with simpler functions and then evaluating the area.

- **Trapezoidal Rule:** Approximates the area under the curve using trapezoids. It is simple, easy to implement, and works well for functions that are nearly linear within subintervals. The formula used is:

$$I \approx \frac{h}{2}\left[f(a) + f(b) + 2\sum_{i=1}^{n-1} f(a+ih)\right]$$

- **Simpson's 1/3 Rule:** Approximates the integral by fitting parabolas through every two subintervals. It provides higher accuracy compared to the trapezoidal rule when the function is smooth. The formula used is:

$$I \approx \frac{h}{3}\left[f(a) + f(b) + 4\sum_{\substack{i=1 \\ i \text{ odd}}}^{n-1} f(a+ih) + 2\sum_{\substack{i=2 \\ i \text{ even}}}^{n-2} f(a+ih)\right]$$

The experiments show that Simpson's Rule generally provides better accuracy than the Trapezoidal Rule for the same number of intervals, since it uses quadratic approximation instead of linear.

Problem Statement-7:-

Solution of Ordinary Differential Equations (ODEs): Implement Euler's method and Runge-Kutta methods (second-order and fourth-order) to solve ODEs.

❒ **Objective:-** The objective of this practical is to implement numerical methods for solving ordinary differential equations (ODEs) of the form

$$\frac{dy}{dx} = f(x, y), \quad y(x_0) = y_0,$$

using Euler's Method and Runge–Kutta Methods of second and fourth order.

Euler's Method is given by

$$y_{n+1} = y_n + hf(x_n, y_n),$$

where h is the step size.

The Second-Order Runge–Kutta Method (RK2) can be expressed as

$$k_1 = hf(x_n, y_n), \quad k_2 = hf(x_n{+}h,\ y_n{+}k_1), \quad y_{n+1} = y_n{+}\tfrac{1}{2}(k_1{+}k_2).$$

The Fourth-Order Runge–Kutta Method (RK4) is defined as

$$\begin{aligned} k_1 &= hf(x_n, y_n), \\ k_2 &= hf(x_n + \tfrac{h}{2},\ y_n + \tfrac{k_1}{2}), \\ k_3 &= hf(x_n + \tfrac{h}{2},\ y_n + \tfrac{k_2}{2}), \\ k_4 &= hf(x_n + h,\ y_n + k_3), \\ y_{n+1} &= y_n + \tfrac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

The aim is to implement these algorithms, compare their numerical solutions with analytical solutions where available, study their

accuracy and stability, and observe the effect of step size on the error.

❒ Algorithm:-

(a) Euler's Method

Algorithm 8 Euler's Method

```
Input x_0, y_0, h, xn
x ← x_0, y ← y_0
while x < xn do
    y ← y + hf(x, y)
    x ← x + h
end while
Return y
```

(b) Runge-Kutta 4th Order

Algorithm 9 Runge-Kutta 4th Order

```
Input x_0, y_0, h, xn
x ← x_0, y ← y_0
while x < xn do
    k_1 = hf(x, y)
    k_2 = hf(x + h/2, y + k_1/2)
    k_3 = hf(x + h/2, y + k_2/2)
    k_4 = hf(x + h, y + k_3)
    y ← y + (k_1 + 2k_2 + 2k_3 + k_4)/6
    x ← x + h
end while
Return y
```

❒ Code:-

```
a) Euler's Method:
#include <stdio.h>

double f(double x, double y) { return x+y; }

int main() {
    double x0, y0, h, xn, x, y;
    printf("Enter x0, y0, h, xn: ");
    scanf("%lf %lf %lf %lf", &x0, &y0, &h, &xn);

    x = x0; y = y0;
```

```
    while(x < xn) {
        y = y + h*f(x,y);
        x += h;
    }
    printf("Approximate y(%lf) = %lf\n", xn, y);
    return 0;
}
```

b) Runge-Kutta 2nd & 4th Order:

```
#include <stdio.h>

double f(double x, double y) { return x+y; }

int main() {
    double x0, y0, h, xn, x, y, k1, k2, k3, k4;
    printf("Enter x0, y0, h, xn: ");
    scanf("%lf %lf %lf %lf", &x0, &y0, &h, &xn);

    x = x0; y = y0;
    while(x < xn) {
        // RK4
        k1 = h*f(x,y);
        k2 = h*f(x+h/2, y+k1/2);
        k3 = h*f(x+h/2, y+k2/2);
        k4 = h*f(x+h, y+k3);
        y = y + (k1+2*k2+2*k3+k4)/6;
        x += h;
    }
    printf("RK4 Approximate y(%lf) = %lf\n", xn, y);
    return 0;
}
```

❒ Output:-

```
Enter x0, y0, h, xn:1
2
3
4
 Approximate y(4.000000) = 11.000000
```

Figure 7.1

```
Enter x0, y0, h, xn:1
2
3
4
 RK4 Approximate y(4.000000) = 60.500000
```

Figure 7.2

❒ **Conclusion:-**

Euler's method is simple and easy to implement but less accurate due to its linear approximation. On the other hand, Runge–Kutta methods (especially RK4) provide much higher accuracy and stability, making them more reliable for practical numerical solutions of differential equations.

Problem Statement-8:-

Curve Fitting: Fit linear, exponential, and polynomial curves to discrete data using the least squares method.

❒ **Objective:-** The objective of this practical is to fit linear, exponential, and polynomial curves to a given set of discrete data points using the Least Squares Method. The method minimizes the sum of squared errors between the observed data and the fitted curve.

For Linear Curve Fitting ($y = a + bx$), the normal equations are

$$\sum y = na + b\sum x, \qquad \sum xy = a\sum x + b\sum x^2.$$

For Exponential Curve Fitting ($y = ae^{bx}$), taking logarithms gives

$$\ln y = \ln a + bx,$$

which reduces to a linear least squares problem.

For Polynomial Curve Fitting ($y = a_0 + a_1x + a_2x^2 + \cdots + a_mx^m$), the normal equations are obtained as

$$\sum x^k y = a_0 \sum x^k + a_1 \sum x^{k+1} + \cdots + a_m \sum x^{k+m},$$
$$k = 0, 1, \ldots, m.$$

The aim is to implement programs for these curve fitting techniques, determine the best-fit parameters, evaluate the goodness of fit, and compare fitted curves with the original data graphically.

❒ Algorithm:-

(a) Linear Fit

Algorithm 10 Linear Curve Fit $y = a + bx$

Compute sums: $\sum x$, $\sum y$, $\sum xy$, $\sum x^2$
Solve normal equations:
$an + b\sum x = \sum y$
$a\sum x + b\sum x^2 = \sum xy$
Return a, b

(b) Exponential Fit

Algorithm 11 Exponential Fit $y = ae^{bx}$

Transform to $\ln y = \ln a + bx$
Compute $\sum x, \sum \ln y, \sum x \ln y, \sum x^2$
Solve normal equations for $\ln a, b$
Return $a = e^{\ln a}, b$

(c) Polynomial Fit (Quadratic)

Algorithm 12 Quadratic Fit $y = a + bx + cx^2$

Compute $\sum x, \sum x^2, \sum x^3, \sum x^4, \sum y, \sum xy, \sum x^2 y$
Solve normal equations:
$an + b\sum x + c\sum x^2 = \sum y$
$a\sum x + b\sum x^2 + c\sum x^3 = \sum xy$
$a\sum x^2 + b\sum x^3 + c\sum x^4 = \sum x^2 y$
Return a, b, c

❒ Code:-

```
a) Linear Fit y = a + bx:
#include <stdio.h>

int main() {
    int n, i;
    double x[20], y[20], sumx=0, sumy=0, sumxy=0, sumx2=0, a, b;
    printf("Enter number of data points: ");
    scanf("%d", &n);
    for(i=0;i<n;i++) {
        scanf("%lf %lf", &x[i], &y[i]);
        sumx+=x[i]; sumy+=y[i];
```

```
        sumxy+=x[i]*y[i]; sumx2+=x[i]*x[i];
    }
    b = (n*sumxy - sumx*sumy)/(n*sumx2 - sumx*sumx);
    a = (sumy - b*sumx)/n;
    printf("Best fit line: y = %.4f + %.4fx\n", a, b);
    return 0;
}
```

b) Exponential Fit → Model:

```
#include <stdio.h>
#include <math.h>

int main() {
    int n, i;
    double x[20], y[20], sumx=0, sumy=0, sumxy=0, sumx2=0;
    double a, b;

    printf("Enter number of data points: ");
    scanf("%d", &n);
    printf("Enter x and y values:\n");
    for(i=0;i<n;i++) {
        scanf("%lf %lf", &x[i], &y[i]);
        sumx += x[i];
        sumy += log(y[i]);   // Taking log of y
        sumxy += x[i]*log(y[i]);
        sumx2 += x[i]*x[i];
    }

    b = (n*sumxy - sumx*sumy) / (n*sumx2 - sumx*sumx);
    a = exp((sumy - b*sumx)/n);

    printf("Best fit exponential curve:
    y = %.4f * e^(%.4f x)\n",
    a, b);
    return 0;
}
```

```
c) Polynomial Fit (Quadratic Example) → Model:
#include <stdio.h>

int main() {
    int n, i;
    double x[20], y[20];
    double sumx=0, sumx2=0, sumx3=0, sumx4=0;
    double sumy=0, sumxy=0, sumx2y=0;
    double a, b, c, D, D1, D2, D3;

    printf("Enter number of data points: ");
    scanf("%d", &n);
    printf("Enter x and y values:\n");
    for(i=0;i<n;i++) {
        scanf("%lf %lf", &x[i], &y[i]);
        sumx += x[i];
        sumx2 += x[i]*x[i];
        sumx3 += x[i]*x[i]*x[i];
        sumx4 += x[i]*x[i]*x[i]*x[i];
        sumy += y[i];
        sumxy += x[i]*y[i];
        sumx2y += x[i]*x[i]*y[i];
    }

    // Normal equations in matrix form:
    // | n     sumx   sumx2 |   | a |   | sumy   |
    // | sumx  sumx2  sumx3 | * | b | = | sumxy  |
    // | sumx2 sumx3  sumx4 |   | c |   | sumx2y |

    D  = n*(sumx2*sumx4 - sumx3*sumx3)
       - sumx*(sumx*sum4 - sumx2*sumx3)
       + sumx2*(sumx*sumx3 - sumx2*sumx2);

    D1 = sumy*(sumx2*sumx4 - sumx3*sumx3)
       - sumx*(sumxy*sumx4 - sumx3*sumx2y)
```

```
        + sumx2*(sumxy*sumx3 - sumx2*sumx2y);

    D2 = n*(sumxy*sumx4 - sumx3*sumx2y)
        - sumy*(sumx*sumx4 - sumx2*sumx3)
        + sumx2*(sumx*sumx2y - sumxy*sumx2);

    D3 = n*(sumx2*sumx2y - sumxy*sumx3)
        - sumx*(sumx*sumx2y - sumxy*sumx2)
        + sumy*(sumx*sumx3 - sumx2*sumx2);

    a = D1/D;
    b = D2/D;
    c = D3/D;

    printf("Best fit quadratic curve:
    y = %.4f + %.4fx + %.4fx^2\n", a, b, c);
    return 0;
}
```

❒ Output:-

```
Enter number of data points:3
1
5
2
3
4
6
 Best fit line: y = 3.5000 + 0.5000x
```

Figure 8.1

```
Enter number of data points:3
 Enter x and y values:
1
2
2
5
3
7
Best fit exponential curve: y = 1.1775 * e^(0.6264 x)
```

Figure 8.2

```
Output
Enter number of data points: 5
Enter x and y values:
1 2
2 5
3 10
4 17
5 26
Best fit quadratic curve: y = 1.0000 + 0.0000x + 1.0000x^2


=== Code Execution Successful ===
```

Figure 8.3

❒ Conclusion:-

Curve fitting using the least squares method provides a systematic way to approximate the relationship between variables. In the case of the linear fit, the best fit line is of the form $y = a + bx$, where a and b are obtained by minimizing the squared errors.

For exponential fitting, the model $y = ae^{bx}$ is used, which becomes linear after taking $\ln y$, allowing the same least squares approach to determine a and b.

Polynomial fitting, such as quadratic fitting, assumes a model of the form $y = a + bx + cx^2$. This captures curvature in the data and is solved using normal equations.

Problem Statement-9:-

System of Linear Equations: Solve a system of linear equations using Gauss elimination or Gauss-Jordan methods.

❒ **Objective:-** The objective of this practical is to solve a system of linear equations $\mathbf{Ax} = \mathbf{b}$ using the Gauss Elimination Method and the Gauss–Jordan Method. These are direct methods based on systematic row operations to reduce the coefficient matrix to a simpler form.

In the Gauss Elimination Method, the system is transformed into an upper triangular system by forward elimination, followed by back substitution:

$$U\mathbf{x} = \mathbf{c},$$

where U is upper triangular.

In the Gauss–Jordan Method, the augmented matrix $[A|b]$ is reduced by row operations into the form

$$[I|x],$$

where I is the identity matrix and $\mathbf{x}$ is the solution vector.

The aim is to implement programs for both methods, apply them to test systems, and compare their efficiency, computational steps, and accuracy.

❒ Algorithm:-

Algorithm 13 Gauss Elimination Method

Form augmented matrix $[A|b]$
for $k = 1$ to $n - 1$ **do**
  for $i = k + 1$ to n **do**
    $m = a_{ik}/a_{kk}$
    for $j = k$ to $n + 1$ **do**
      $a_{ij} = a_{ij} - m \cdot a_{kj}$
    end for
  end for
end for
Back substitution:
$x_n = a_{n,n+1}/a_{nn}$
for $i = n - 1$ down to 1 **do**
  $x_i = (a_{i,n+1} - \sum_{j=i+1}^{n} a_{ij}x_j)/a_{ii}$
end for
Return solution vector x

❒ Code:-

```
#include <stdio.h>
#define N 3

int main() {
    float a[N][N+1], x[N];
    int i,j,k;
    float ratio;

    printf("Enter augmented matrix (%dx%d):\n", N, N+1);
    for(i=0;i<N;i++)
        for(j=0;j<=N;j++)
            scanf("%f",&a[i][j]);

    for(i=0;i<N-1;i++){
        for(j=i+1;j<N;j++){
            ratio=a[j][i]/a[i][i];
            for(k=0;k<=N;k++) a[j][k]-=ratio*a[i][k];
        }
    }

    x[N-1]=a[N-1][N]/a[N-1][N-1];
```

```
    for(i=N-2;i>=0;i--){
        x[i]=a[i][N];
        for(j=i+1;j<N;j++) x[i]-=a[i][j]*x[j];
        x[i]/=a[i][i];
    }

    printf("Solution:\n");
    for(i=0;i<N;i++) printf("x[%d]=%f\n",i+1,x[i]);
    return 0;
}
```

❒ Output:-

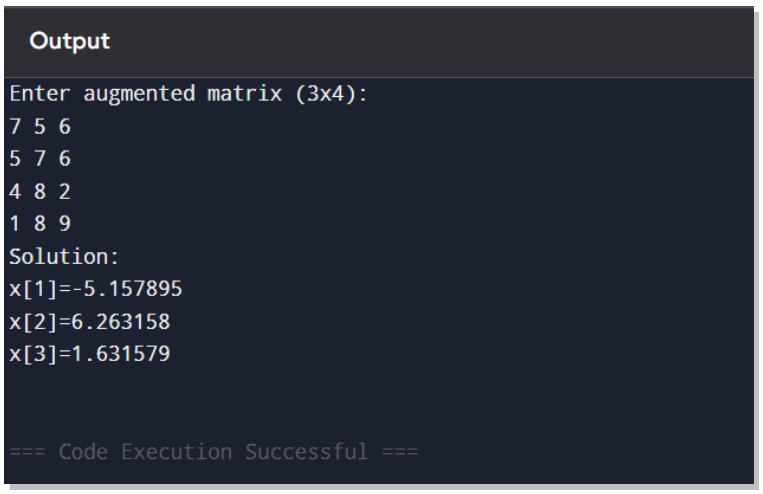

Figure 9

❒ Conclusion:-

The Gauss Elimination method reduces a system of linear equations

$$\begin{aligned}
&a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1,\\
&a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2,\\
&\ldots, \quad a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n
\end{aligned}$$

into an upper triangular system through forward elimination.

The back substitution step then solves for the unknowns starting

from

$$x_n = \frac{a_{n,n+1}}{a_{nn}},$$

and recursively,

$$x_i = \frac{1}{a_{ii}} \left(a_{i,n+1} - \sum_{j=i+1}^{n} a_{ij} x_j \right), \quad i = n-1, n-2, \ldots, 1.$$

This approach is efficient for small to medium size systems and ensures exact solutions in the absence of round-off errors or singularities.

Acknowledgment

I would like to express my deepest sense of gratitude to my respected teacher, **Mr. Amitava Sardar**, for his valuable guidance, constant encouragement, and continuous support throughout the completion of this Computational Mathematics practical work. His clear and systematic teaching, patient explanations of difficult concepts, and insightful suggestions have not only enhanced my understanding of the subject but have also inspired me to approach problems with greater clarity and confidence. Without his encouragement and dedicated guidance, this work would not have been possible.

I am also sincerely grateful to the **Department of Computer Science** of my college for providing me with the facilities, resources, and academic environment that made this work possible. The availability of proper infrastructure, learning materials, and an atmosphere of academic curiosity has played a vital role in shaping my knowledge and practical skills in computational methods.

I would like to take this opportunity to extend my heartfelt thanks to my **classmates and friends**, who have always been supportive and cooperative throughout the preparation of this record. Their discussions, feedback, and encouragement have helped me remain motivated and complete this work successfully.

Finally, I wish to thank everyone who directly or indirectly contributed to the successful completion of this practical work.

Thank you,

-.-.-.